https://github.com/TSika-1/CS352-Unit1-Assignment2

After changing from an ArrayList to a LinkedList, I tested that the program still behaves the same. I ran the 5-contestant round with printing turned on and checked each step by hand against the table from Unit 1. Correct answers moved a contestant to the front, incorrect answers moved them to the end, and the line always had 5 contestants with no duplicates or missing names. I then ran 8 contestants for 12 rounds and checked the average positions. They were all close to the middle value of 3.5, so no contestant got stuck at one end. Because the output and fairness results matched the ArrayList version, I concluded the LinkedList version works correctly.

Step 4: Timing tables

Timing table so far

Contestants	Rounds	ArrayList (ms)	LinkedList (ms)
43	10	0	0
43	100	1	1
43	1,000	9	11
43	10,000	28	23
45,000	10	1,793	29,389
45,000	100	17,165	296,547
45,000	1,000	168,493	*(running)*
45,000	10,000	*(your number)*	*(too long)*

With 43 contestants, the two lists performed almost the same. For 10,000 rounds the ArrayList took 28 ms and the LinkedList took 23 ms, so the list size was too small for the type to matter. With 45,000 contestants, the LinkedList was much slower. For 10 rounds it took 29,389 ms compared to 1,793 ms for the ArrayList, and for 100 rounds it took 296,547 ms compared to 17,165 ms. That makes it about 16 to 17 times slower. I think this happens because my algorithm uses get(index) and remove(index) for every contestant. A LinkedList can’t jump to a position, so it has to follow the links one node at a time from the start or the end. An ArrayList can go straight to the position, and shifting its elements is done with fast memory copying. A LinkedList is fast at adding to the front or the end, but my algorithm spends most of its time reaching the middle of the list, so that benefit doesn’t help. Because of this, I would recommend

the ArrayList for this program. A LinkedList would only make sense if the program used an iterator that remembers its position, so it wouldn't have to search from the start each time.

For my Unit 1 idea of swapping contestants in the same list, I expect the LinkedList to take more time than the ArrayList. Swapping uses get() and set() by index, which are very fast in an ArrayList but require walking through nodes in a LinkedList. With 45,000 contestants, that would make the LinkedList much slower again.

What did I learn? The type of list changes how long the same algorithm takes, even when the code is nearly identical.
What did I like? It was a very small code change, and it was interesting to measure the difference.
What was confusing? Why a LinkedList isn't faster when adding to the front is fast, and how long the large runs take.
What would I add? An iterator-based version for the LinkedList, or runs that average several timings.